

PyTrackingAnalysis — User Guide

Table of contents

- 1. Overview
- 2. Environment setup with uv
 - Windows
 - macOS
 - Linux
- 3. Project directory structure
- 4. The tracking_config.yaml reference
- 5. The desktop UI
 - Launching the apps
 - Analysis Hub
 - Config Editor
 - QC Viewer
- 6. Running the pipeline from a notebook or script
- 7. Understanding the outputs
- 8. Batch analysis across multiple experiments
- 9. Quick reference

Scripts (saved analysis recipes) and the visual Script Editor have their own dedicated guide: [scripts_guide.md](#).

1. Overview

PyTrackingAnalysis is a Python pipeline for analysing insect-tracking data exported from DTrack. A single configuration file (`tracking_config.yaml`) describes the experiment — the tracking hardware, the experimental design, and how each physical tracking region maps to a treatment group. From that one file the pipeline can produce summary CSVs, statistical comparisons, publication-quality plots, and a multi-page PDF report.

There are three ways to drive the pipeline:

Interface	Best for
Analysis Hub (<code>pytrack-hub</code>)	Day-to-day use; loads experiments, runs analyses, shows plots in a tabbed dock
Config Editor (<code>pytrack-config</code>)	Authoring <code>tracking_config.yaml</code> + visual Script Editor for saved recipes
QC Viewer (<code>pytrack-qc</code>)	Per-tracker data-quality tables + XY / distance / timeline plots
Jupyter notebook (<code>Notebooks/SimpleTracker.ipynb</code>)	Exploratory work; custom plots

Interface	Best for
Python script / REPL	Automation; integration with other tools

2. Environment setup with uv

uv is a fast Python package manager that reads `pyproject.toml` and locks exact dependency versions. The project requires **Python 3.13 or later**.

2.1 Windows

Install uv

Open PowerShell and run:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1  
| iex"
```

Restart the terminal so **uv** is on **PATH**.

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git  
cd PyTrackingAnalysis  
  
# Create a virtual environment using the Python version in .python-version  
(3.13)  
uv sync
```

uv sync reads `pyproject.toml`, downloads and installs all dependencies into `.venv\`, and is idempotent — safe to run again after pulling updates.

Activate the environment (optional — **uv run** works without it)

```
.venv\Scripts\Activate.ps1
```

Run the UI

```
# With the environment activated:  
pytrack                # Analysis Hub (default entry point)  
pytrack-hub            # Analysis Hub (same as pytrack)  
pytrack-config         # Config Editor  
pytrack-qc            # QC Viewer  
  
# Or without activating, using uv run:
```

```
uv run pytrack-hub

# With an explicit project directory:
uv run pytrack-hub "C:\Users\you\Experiments\Trial1"
```

PyQt6 note on Windows: If a "platform plugin not found" error appears, install the Visual C++ Redistributable from Microsoft's website, then retry.

2.2 macOS

Install uv

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

Restart the terminal (or run `source ~/.zshrc` / `source ~/.bash_profile`).

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
# With environment activated:
pytrack                # Analysis Hub (default entry point)
pytrack-hub            # Analysis Hub (same as pytrack)
pytrack-config         # Config Editor
pytrack-qc             # QC Viewer

# Without activating:
uv run pytrack-hub

# With a project directory:
uv run pytrack-hub /path/to/Trial1
```

macOS display server note: PyQt6 requires a display. Confirm XQuartz is not needed (native macOS Cocoa backend is used automatically).

2.3 Linux

Install uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
# Or via pip if curl is unavailable:
pip install uv
```

Restart the terminal or run `source ~/.bashrc`.

Install system Qt dependencies (if missing)

On Debian/Ubuntu:

```
sudo apt install libxcb-cursor0 libxcb-xinerama0 libxkbcommon-x11-0
```

On Fedora/RHEL:

```
sudo dnf install xcb-util-cursor libxkbcommon-x11
```

Clone the repository and create the environment

```
git clone https://github.com/your-org/PyTrackingAnalysis.git
cd PyTrackingAnalysis
uv sync
```

Activate the environment (optional)

```
source .venv/bin/activate
```

Run the UI

```
pytrack-hub                # Analysis Hub
uv run pytrack-hub          # works from any directory
uv run pytrack-hub /path/to/Trial1 # open a specific project
uv run pytrack-config       # Config Editor
uv run pytrack-qc /path/to/Trial1 # QC Viewer
```

Headless / SSH note: PyQt6 requires an X11 or Wayland display. If running over SSH, forward the display with `ssh -X` or use a VNC/RDP session. For fully headless batch processing, use the Python

API (§6) instead.

Updating the environment

After pulling new commits:

```
git pull
uv sync          # installs / removes packages to match the updated lock
file
```

3. Project directory structure

A **project directory** is the root folder for a single experiment. It must contain `tracking_config.yaml` and a `data/` sub-folder with the DTrack export files. The pipeline creates `analysis/` and `qc/` automatically on first run.

```
MyExperiment/                                     ← project directory (pass this to the
UI)
├── tracking_config.yaml                         ← experiment configuration (required)
├── data/                                       ← DTrack export files (required)
│   ├── ExperimentName.xlsx                   ← main DTrack workbook
│   ├── ExperimentName_Data_1.csv             ← per-tracker CSV, one per tracking
region
│   ├── ExperimentName_Data_2.csv
│   ├── ...
│   ├── ExperimentName_BG.jpg                 ← background image (optional)
│   ├── LeftProgram.txt                      ← stimulus program files (optional)
│   └── RightProgram.txt
├── analysis/                                ← created by the pipeline
│   ├── ExperimentName_experiment_summary.txt
│   ├── ExperimentName_Summary.csv
│   ├── ExperimentName_Summary_Facet.csv
│   ├── ExperimentName_Stats.txt
│   ├── ExperimentName_report.pdf
│   ├── ExperimentName_plot_pi_facet.png
│   ├── ExperimentName_plot_percentage_facet.png
│   ├── ExperimentName_plot_transitions_facet.png
│   └── ExperimentName_plot_totaldistance_facet.png
└── qc/                                       ← created by the pipeline
    └── ExperimentName_data_quality.csv
```

Naming rules:

- The `.xlsx` file determines the experiment name. Every output file is prefixed with that name.
- The per-tracker CSVs must follow the pattern `<name>_Data_<N>.csv`. `N` is matched to tracking region `T_N-1` in the YAML (i.e. `_Data_1.csv` → region `T_0`).

- `tracking_config.yaml` must be at the **top level** of the project directory — not inside `data/`.

Batch layout (multiple experiments)

For batch analysis, place multiple project directories under a common parent:

```
AllExperiments/                                     ← select this as the project dir in
Batch experiments mode
├── Trial1/
│   ├── tracking_config.yaml
│   └── data/ ...
├── Trial2/
│   ├── tracking_config.yaml
│   └── data/ ...
└── Trial3/
    ├── tracking_config.yaml
    └── data/ ...
```

Each sub-directory is processed independently. Directories that don't qualify are skipped: the Hub's batch mode reports each skip in the Output and Errors tabs, while the Python `batch_analyze()` helper skips quietly. See §8 for the full batch procedure.

4. The `tracking_config.yaml` reference

`tracking_config.yaml` has four possible top-level sections: `global`, `tracking_regions`, `counting_regions`, and `scripts`. `global` and `tracking_regions` are required for every experiment; `counting_regions` is required for the two-choice and counter tracking types; `scripts` is optional.

Creating a valid file

There are three equally valid ways to create one:

1. **Config Editor** (recommended) — `pytrack-config` gives you structured forms for every section, bulk region generation, and a live YAML preview, so the file is valid by construction.
2. **Copy an existing config** — copy a working `tracking_config.yaml` into the new project directory and edit it. The Hub's **Batch tools** → **Copy YAML** can push one file into every sub-directory of a batch parent.
3. **Write it by hand** — any text editor works; the file is plain YAML.

Rules that make a file *valid*:

- The file must be named `tracking_config.yaml` and live in the project directory (next to `data/`, not inside it). Batch mode matches the name case-insensitively; everywhere else use the exact lowercase name.
- An experimental design is **required** — an experiment will refuse to load without a parseable config.
- `tracking_type` must be one of the values in §4.1 (only a *missing* key falls back to `TRACKER`; a wrong value is an error).

- `TWOCHOICETRACKER` and `TWOCHOICECOUNTER` must define **exactly two** `counting_regions` entries.
- Every `counting_regions` entry must have an `alias` key.

Check a file at any time with the Hub's **Tools** → **Validate YAML** button — parse errors are reported in the Output and Errors tabs.

4.1 `global` — required fields

```
global:
  tracking_type: TWOCHOICETRACKER    # see table below
  tracking_rig:  colosseum           # see table below
```

`tracking_type`

Selects the analysis mode. Choose the one that matches how DTrack recorded your data. The value is upper-cased before matching, so `twochoicetracker` works too. If the key is missing entirely, `TRACKER` is assumed; an unrecognized value is an error that lists the valid choices.

Value	Description
TRACKER	Position tracking only — computes distance and speed
TWOCHOICETRACKER	Two-region choice assay — computes PI, percentage time, transitions
XCHOICETRACKER	Multi-region assay along a linear axis — computes adjusted X position
PAIRWISEINTERACTIONTRACKER	Proximity-based interaction scoring between pairs of tracked animals
COUNTER	Frame-count based occupancy (no continuous position)
TWOCHOICECOUNTER	Counter-based two-region choice — PI and percentage
PAIRWISEINTERACTIONCOUNTER	Counter-based pairwise interaction scoring

`tracking_rig`

Selects the hardware calibration preset. `fps` and `mm_per_pixel` are set automatically from the preset; all other parameters use the defaults shown below. The value is matched case-insensitively with spaces/hyphens treated as underscores (`Arena Max` → `arena_max`), and the common misspelling `colloseum` is accepted for `colosseum`. An unknown rig name is not an error — the tracking type is still applied but all calibration values stay at their generic defaults, so double-check the spelling.

Value	mm per pixel	Notes
small_arena	0.056	

Value	mm per pixel	Notes
arena_max	0.145	
colosseum	0.108	
obscura	0.131	
movie	—	You must supply <code>fps</code> and <code>mm_per_pixel</code> manually

`fps` for all hardware rigs is read from the timestamps in the DTrack export rather than set to a fixed value. Only the `movie` rig requires an explicit `fps`.

4.2 `global` — experimental design

```
global:
  experimental_design_factors:
    feeding: [Starved, Control]
    sex:     [Male, Female]
```

List every factor and its levels. These are used for axis labels and plot titles. The actual assignment of factors to each physical tracking region is made in `tracking_regions` (see §4.4).

4.3 `global` — optional fields

```
global:
  # Split the recording into time phases for faceted plots and statistics.
  # [10, 70] creates three phases: 0-10 min, 10-70 min, 70+ min.
  # Remove this key entirely to disable faceted analysis.
  facet_cutoffs: [10, 70]

  # Only needed for the 'movie' rig, or to override a hardware preset.
  fps: 30
  mm_per_pixel: 0.108

  # Smoothing window for speed calculation (seconds). Default: 1
  speed_window_seconds: 1

  # Speed range [min, max] mm/s that defines micro-movement. Default: [0.2, 2]
  micromove_speed_mm_sec: [0.2, 2]

  # Minimum speed (mm/s) to count as walking. Default: 2
  walking_speed_mm_sec: 2

  # Minimum continuous resting duration (minutes) to count as sleep.
  # Default: 5
  sleep_threshold_min: 5
```



```
# Distance thresholds (mm) for interaction detection.
# Only used with PAIRWISEINTERACTIONTRACKER / PAIRWISEINTERACTIONCOUNTER.
interaction_distances: [8]
```

How overrides are interpreted: the rig preset is applied first, then any of the recognized parameter keys present in `global`: override the preset value. Exactly these keys are recognized as overrides — `fps`, `mm_per_pixel`, `speed_window_seconds`, `micromove_speed_mm_sec`, `walking_speed_mm_sec`, `sleep_threshold_min`, `interaction_distances`. Any other key in `global`: is carried along but ignored by the parameter system, so a typo like `walking_speed` will not error — it simply won't take effect.

`facet_cutoffs` is read separately (not a parameter override): it becomes the default for every faceted plot, summary, and statistics run, and is inherited by script steps whose own `cutoffs` field is blank (see [scripts_guide.md](#)).

4.4 tracking_regions

One entry per DTrack tracking region. Region names must match the naming scheme used in the CSV files: region `T_0` corresponds to `_Data_1.csv`, `T_1` to `_Data_2.csv`, and so on.

```
tracking_regions:
  T_0:
    experimental_factors: Starved, Female # comma-separated, order must
    match                                                         # experimental_design_factors
    order
    x_location_multiplier: 1 # set to -1 to flip the X axis
    y_location_multiplier: 1 # set to -1 to flip the Y axis
  T_1:
    experimental_factors: Control, Male
    x_location_multiplier: 1
    y_location_multiplier: 1
# ... one entry per region
```

How each field is interpreted:

- `experimental_factors` — a free-form string that becomes the region's *Treatment* label used in grouping, plots, and statistics. For multi-factor designs list the levels comma-separated in the same order as `experimental_design_factors` (e.g. `Starved, Female`). Regions sharing the same string are analysed as one group. A missing key means the region has an empty treatment (it still loads, but groups as blank — the experiment summary lists such regions as *(unassigned)*).
- `x_location_multiplier` / `y_location_multiplier` — correct for physical differences in camera orientation between regions. Use `1` for no correction and `-1` to mirror an axis. Only `1` and `-1` are meaningful: any other value (or a missing key) is silently treated as `1`.

The experiment summary text file (§7) includes a formatted description of the loaded design — factors, region assignments, non-unit multipliers, counting regions, and cutoffs — so you can verify the YAML was interpreted the way you intended.

4.5 counting_regions

Maps the region labels used inside the DTrack data to canonical choice names. Required for the counter types (**COUNTER**, **TWOCHOICECOUNTER**, **PAIRWISEINTERACTIONCOUNTER**) and for **TWOCHOICETRACKER** — both two-choice types are validated to have **exactly two** counting regions and will refuse to load otherwise.

```
counting_regions:
  Light:
    alias: Light, LL, L      # any of these strings in the data = "Light"
  NoLight:
    alias: NoLight, NL, N
```

How it is interpreted: each key (**Light**, **NoLight**) is a canonical characteristic name; **alias** is a comma-separated list of the raw region labels that map to it (whitespace around each alias is stripped). Matching is exact and case-sensitive after stripping. Every entry **must** have an **alias** key — omitting it is a config error that prevents the design from loading.

4.6 scripts — saved analysis recipes (optional)

Saved, re-runnable step lists that the Hub's **Scripts** card and batch mode execute. Normally you author these visually in the Script Editor rather than by hand:

```
scripts:
- name: nightly      # free-form, except 'batch' which is special (§8)
  steps:
  - action: load_experiment
    params: {path: '.', force_preprocessing: false}
  - action: run_analysis
    params: {facet: true, cutoffs: ''}
```

Each script is **{name, steps}**; each step is **{action, params}** where **action** is one of the registered action keys and **params** matches that action's schema. A script named **batch** (case-insensitive) is what **Batch experiments** mode runs in each sub-folder. See [scripts_guide.md](#) for the full action reference, validation rules, and hand-editing guidance.

4.7 Complete minimal example

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    genotype: [WT, Mutant]
    facet_cutoffs: [10, 60]

tracking_regions:
  T_0:
    experimental_factors: WT
    x_location_multiplier: 1
    y_location_multiplier: 1
  T_1:
    experimental_factors: Mutant
    x_location_multiplier: 1
    y_location_multiplier: 1
```

5. The desktop UI

PyTrackingAnalysis ships three independent PyQt6 apps, each launched as its own window. They share a common pyflic-style theme (category-colored cards, top bar, PlotDock) so the visual language is consistent across all three.

Command	Window	Purpose
pytrack-hub (or just pytrack)	Analysis Hub	Day-to-day driver — loads experiments, runs single / batch analyses, renders figures in a tabbed dock, launches Config + QC
pytrack-config	Config Editor	Structured editor for tracking_config.yaml + visual Script Editor for saved recipes
pytrack-qc	QC Viewer	Per-tracker data-quality table + XY / distance / quality-timeline plots

5.1 Launching the apps

```
# With the environment active:
pytrack                                # Hub (shorthand)
pytrack-hub                            # Hub
pytrack-hub /path/to/MyExperiment     # Hub, pre-loaded project

pytrack-config                        # Config Editor (opens last-used
or ./tracking_config.yaml)
pytrack-config /path/to/MyExperiment  # Config Editor, pre-loaded
project YAML

pytrack-qc /path/to/MyExperiment      # QC Viewer, pre-loaded project

# Without activating, through uv:
```

```
uv run pytrack-hub
uv run pytrack-config /path/to/Trial1

# Dev shortcut (equivalent to the console scripts above):
python -m pytrackinganalysis hub /path/to/Trial1
python -m pytrackinganalysis config /path/to/Trial1
python -m pytrackinganalysis qc /path/to/Trial1
```

Desktop launcher / taskbar icon (Linux). The apps set their own window icon (a fly in a tracking reticle), but on Wayland/GNOME the *taskbar* icon comes from a `.desktop` entry. Install entries for all three apps once per environment with:

```
uv run pytrack-install-desktop
```

This also adds the apps to your desktop's application launcher. Re-run it if you move the project or recreate `.venv` (the entries embed absolute paths).

All three apps persist the light/dark theme choice to `~/.config/pytrackinganalysis/ui.json`. Recent projects are tracked there too.

5.2 Analysis Hub (`pytrack-hub`)

Six cards, each on a sidebar entry:

- **Project** — pick the experiment folder (the text box shows just the folder name to stay readable; hover it for the full path), choose a YAML config, launch the Config Editor or QC Viewer in their own windows, and **Reload** to re-scan the folder.
- **Load** — radio toggle between **Single project** and **Batch experiments** (the round ? button next to the radio opens an in-app explanation of the batch procedure — see §8). In single mode the button reads **Load experiment** and caches the Experiment so subsequent analyses re-use the parsed data; in batch mode it becomes **Run batch script** and runs the script named `batch` in every sub-folder.
- **Analyze** — **Run Analysis**, **Run QC only**, **Create PDF Report** (single mode only). All tasks run on a background thread; stdout/stderr streams to the **Output** tab in real time.
- **Plots** — dynamically populated with the faceted plots valid for the loaded tracking type (`plot_pi_facet`, `plot_totaldistance_facet`, etc.). Each click adds a new tab to the PlotDock. Toggle **Interactive plots** in the top bar to switch between static PNG rendering (fast) and a live canvas with pan / zoom / save toolbar.
- **Scripts** — lists saved analysis recipes from the active YAML's `scripts` section. **Run Script / Run All** executes them and routes each step's log output to the Output tab and each figure to a PlotDock tab. Author scripts from the Config Editor (see 5.3 and [the scripts guide](#)).
- **Tools** — validate YAML, open the `analysis/` or `qc/` folder in the system file browser, open the **Batch tools** dialog (convert sub-directory layouts, bulk-rename sub-directories, copy a YAML into every sub-directory, combine summary CSVs across sub-directories), and clear the matplotlib cache.

The plot dock (right-hand side)

- The first tab is always **Output** — the chronological log of everything the Hub does.
- The second tab is **Errors** — a permanent tab that collects only warnings and errors (failed tasks with tracebacks, skipped batch sub-folders, YAML validation problems, dismissed warning pop-ups, ...) so they can't get lost in the normal output. When issues arrive while you're on another tab, the tab title shows an unseen count, e.g. **Errors (3)**; viewing the tab resets it.
- Every plot or artifact opens as an additional closable tab. The **Clear plots** button in the tab-bar corner closes all of them at once and returns to the Output tab (Output and Errors are never closed).

5.3 Config Editor (**pytrack-config**)

Structured editor for **tracking_config.yaml** with three tabs wrapped in a Card:

- **Global** — drop-downs for **tracking_type** and **tracking_rig**; a table for experimental-design factors; optional facet cutoffs; text fields for each parameter override.
- **Tracking regions** — one row per region. **Generate N regions** bulk-fills **T₀...T_(N-1)**. X/Y multipliers are restricted to **1 / -1**.
- **Counting regions** — treatment label → DTrack aliases.

A **YAML preview** Card below the tabs renders the live serialization for trust / debugging; an amber ● dirty indicator surfaces unsaved edits.

Visual Script Editor

Open via the scripts icon in the top bar. A non-modal window with three panes:

- **Palette** (left) — category-grouped tile list of the registered actions (**load_experiment**, **filter_by_quality**, **filter_by_region**, **run_qc**, **run_analysis**, **summarize**, **run_pairwise_comparisons**, **create_report**, plus one plot action per plot valid for the project's tracking type). Double-click to add a step.
- **Canvas** (center) — ordered step cards with the action's icon, a parameter-summary chip, and move-up / move-down / delete buttons.
- **Inspector** (right) — dynamic form for the selected step with widgets derived from the action's param schema (spinbox, combo, line edit, checkbox, browse-for-path, comma-list). Red banner reports validation errors inline.
- **Preview** (bottom) — live YAML of the current script.

Scripts are stored under the **scripts:** key of the surrounding **tracking_config.yaml**. The Hub's **Scripts** card reads the same file, so saving in the Script Editor makes scripts immediately runnable from the Hub. A script named **batch** has a special role in batch mode (§8).

Full documentation — every action, its parameters, faceting rules, and the **batch special case — lives in [scripts_guide.md](#).**

5.4 QC Viewer (**pytrack-qc**)

- Left pane — **Trackers** table with columns **Tracker**, **HighQuality**, **NotFound**, **Indiscernible**, **StartMinutes**, **EndMinutes**. Rows auto-tint green ($\geq$ cutoff) or red ($<$ cutoff) based on the **qc_cutoff** spinbox; a filter box narrows by tracker name.

- Right pane — **PlotDock** that auto-populates when you select a tracker with four tabs:
 - **XY trajectory** — RelX/RelY scatter coloured by time (viridis).
 - **Total distance over time** — cumulative **Dist_mm** vs **Minutes**.
 - **X / Y vs time** — stacked RelX(t), RelY(t) line plots.
 - **Data quality timeline** — per-frame **DataQuality** category plotted as a time series so bad-tracking regions jump out visually.
- **Export data_quality.csv** writes the full table to disk for external review.

6. Running the pipeline from a notebook or script

The analysis pipeline is also available as a Python API. This is the approach used in [Notebooks/SimpleTracker.ipynb](#).

Setup

```
import warnings
warnings.filterwarnings("ignore")

# PyTrackingAnalysis is installed as a package by `uv sync`, so import it
# by its
# full dotted path. This works from any working directory – no os.chdir
# needed.
from pytrackinganalysis.Experiment import Experiment, batch_analyze
```

Single experiment

```
# Pass the project directory – the one containing tracking_config.yaml and
# data/
exp = Experiment("./Data/Trial1/")

# Human-readable summary of the loaded experiment
print(exp)

# Detailed per-tracker overview (also saved to analysis/)
exp.experiment_summary()

# Data quality report (also saved to qc/)
exp.qc(cutoff=0.9)      # cutoff = minimum fraction of valid frames per
# tracker

# Save summary CSVs (flat + faceted) to analysis/
exp.save_summary()

# Run pairwise statistical comparisons and save results to analysis/
exp.stats()

# Save plots to analysis/ as PNG files
exp.save_plots()
```

```
# Build a multi-page PDF report in analysis/
exp.create_report()

# — OR run the complete pipeline in one call:
_____

exp.run_analysis()      # summary → qc → save_summary → save_plots → stats
exp.create_report()     # PDF report (separate call so you can skip it)
```

Accessing individual plots interactively

Inside a Jupyter notebook with `%matplotlib inline`:

```
# These display the plot inline in the notebook (not saved to disk).
exp.arena.plot_pi()
exp.arena.plot_pi_facet(cutoffs=[10, 70])
exp.arena.plot_percentage_facet(cutoffs=[10, 70])
exp.arena.plot_transitions_facet(cutoffs=[10, 70])
exp.arena.plot_totaldistance_facet(cutoffs=[10, 70])
```

The available plot methods depend on `tracking_type`:

tracking_type	Plot methods
TRACKER	plot_totaldistance_facet
TWOCHOICETRACKER	plot_pi_facet, plot_percentage_facet, plot_transitions_facet, plot_totaldistance_facet
TWOCHOICECOUNTER	plot_pi_facet, plot_percentage_facet
XCHOICETRACKER	plot_adjusted_x_position_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONTRACKER	plot_interactions_facet, plot_totaldistance_facet
PAIRWISEINTERACTIONCOUNTER	plot_interactions_facet

Batch processing from a script

```
results = batch_analyze("./Data/")

for path, status in results.items():
    tag = "OK " if status == "ok" else "FAIL"
    print(f" {tag} {path}")
    if status != "ok":
        print(f"      {status}")
```

`batch_analyze` scans every immediate sub-directory of the supplied path, identifies valid experiment directories, runs `exp.run_analysis()` and `exp.create_report()` on each, and returns a `{path: "ok" | error_message}` dictionary.

7. Understanding the outputs

All outputs are written relative to the project directory.

analysis/ — main results

File	Contents
*_experiment_summary.txt	Rig settings, parameters, a formatted description of the experimental design (factors, region assignments, non-unit multipliers, counting regions, cutoffs), data quality overview, per-tracker table
*_Summary.csv	Per-tracker summary statistics (one row per tracker)
*_Summary_Facet.csv	Same, split into the time phases defined by <code>facet_cutoffs</code>
*_Stats.txt	Pairwise statistical comparisons (Mann-Whitney U) across treatment groups
plot.png	One PNG per plot type, named after the plot method
*_report.pdf	Multi-page PDF: experiment summary → QC table → tracker grid plots → all plots

qc/ — data quality

File	Contents
*_data_quality.csv	Per-tracker fraction of valid (non-missing) frames; trackers below <code>cutoff</code> are flagged

8. Batch analysis across multiple experiments

There are two ways to process many experiments in one go: **Batch experiments mode in the Hub**, which runs a *script* of your choosing per sub-folder, and the Python `batch_analyze()` function, which runs the fixed full pipeline per sub-folder.

Layout requirement (both methods)

Each experiment sub-directory must be a self-contained project directory under one common parent:

ParentFolder/
directory
├─ Experiment_A/
│ └─ tracking_config.yaml
UI batch mode)

← select this as the project

← required (with a 'batch' script for


```

├── data/
│   ├── Experiment_A.xlsx      ← required
│   └── Experiment_A_Data_*.csv
└── Experiment_B/
    ├── tracking_config.yaml
    └── data/ ...

```

Each sub-directory uses its **own** `tracking_config.yaml`, so different experiments can have different tracking types, rigs, designs, and batch scripts.

8.1 Batch experiments mode in the Hub

This is the flexible method: *you* decide what runs in each sub-folder by authoring a script named `batch` in each sub-folder's YAML (Script Editor → new script → name it `batch`; the name is matched case-insensitively). The round ? button next to the radio in the Load card shows this same procedure in-app.

Step by step:

1. In the **Project** card, browse to `ParentFolder/` (the *parent*, not one of the experiments).
2. In the **Load** card, select **Batch experiments**. The load button relabels to **Run batch script**; the single-project analysis buttons and the Scripts card grey out since they don't apply.
3. Click **Run batch script**. For every immediate sub-directory (processed in sorted order) the Hub:
 - looks for `tracking_config.yaml` (case-insensitive). Missing → the sub-folder is **skipped** with a warning;
 - reads its `scripts:` list. Unreadable YAML → **skipped**, counted as failed;
 - finds the script named `batch` (case-insensitive). Absent → **skipped** with a warning;
 - runs that script with the sub-directory as its project dir. A `load_experiment` step with `path: "."` therefore loads *that* sub-folder's data.
4. When the run finishes, all per-folder log lines flush to the **Output** tab, every skip/failure also lands in the **Errors** tab, any figures the scripts produced open as plot tabs, and a summary line reports the counts: `Batch script complete: N ran, N without 'batch' script, N without config, N failed (of N subdirs)`.

Because scripts start from a clean slate in each folder, a `batch` script **must begin with a `load_experiment` step** (leave its path as `.`). A typical `batch` script is: `load_experiment` → `filter_by_quality` → `run_analysis` → `create_report`.

An error inside one sub-folder's script never stops the others — it is logged, counted as failed, and the run moves on.

Preparing many folders at once: the Hub's **Tools** → **Batch tools** dialog can copy one master YAML (including its `batch` script) into every sub-directory, bulk-rename sub-directories, convert flat layouts into the `data/` structure, and afterwards combine every `*_Summary.csv` / `*_Summary_Facet.csv` across sub-folders into one CSV per type tagged by sub-directory name.

8.2 Fixed pipeline from Python

```
from pytrackinganalysis.Experiment import batch_analyze

results = batch_analyze("./ParentFolder/") # {path: 'ok' | error message}
```

`batch_analyze` runs `run_analysis()` + `create_report()` on every immediate sub-directory that contains a `tracking_config.yaml` and a `data/` folder with at least one `.xlsx`; other directories are skipped. Optional arguments: `cutoffs` (override every experiment's facet cutoffs), `qc_cutoff` (default 0.9), and `force_preprocessing`. No `batch` script is involved — use this when every experiment should get the identical standard pipeline.

Results

Either way, results are written into each experiment's own `analysis/` and `qc/` folders (e.g. `Experiment_A/analysis/Experiment_A_report.pdf`) — batch runs never mix outputs across experiments.

9. Quick reference

Starting the UI

```
# Analysis Hub (the default entry point):
pytrack                               # shorthand for pytrack-hub
pytrack-hub /absolute/path/to/project

# Via uv (works from any directory, no activation needed):
uv run pytrack-hub /absolute/path/to/project

# Standalone Config Editor (with visual Script Editor):
pytrack-config
pytrack-config /path/to/project

# Standalone QC Viewer:
pytrack-qc /path/to/project

# One-time: install launcher entries + taskbar icon (Linux):
pytrack-install-desktop
```

Minimal tracking_config.yaml

```
global:
  tracking_type: TWOCHOICETRACKER
  tracking_rig: colosseum
  experimental_design_factors:
    treatment: [A, B]

tracking_regions:
```

```
T_0:
  experimental_factors: A
  x_location_multiplier: 1
  y_location_multiplier: 1
T_1:
  experimental_factors: B
  x_location_multiplier: 1
  y_location_multiplier: 1
```

One-line pipeline (Python)

```
from pytrackinganalysis.Experiment import Experiment
exp = Experiment("/path/to/project/")
exp.run_analysis()
exp.create_report()
```

Rig calibration values

tracking_rig	mm / pixel
small_arena	0.056
arena_max	0.145
colosseum	0.108
obscura	0.131
movie	user-supplied

Default analysis parameters

Parameter	Default	YAML key
Speed smoothing window	1 second	speed_window_seconds
Micro-movement range	0.2 – 2 mm/s	micromove_speed_mm_sec
Walking threshold	2 mm/s	walking_speed_mm_sec
Sleep threshold	5 min continuous rest	sleep_threshold_min
Interaction distance	8 mm	interaction_distances

Environment commands

```
uv sync          # install / update all dependencies
uv add <package> # add a new dependency
uv run python <script> # run without activating the environment
```

```
source .venv/bin/activate # activate on macOS / Linux  
.venv\Scripts\Activate.ps1 # activate on Windows PowerShell
```